

Classification

June 14, 2026

1 Classification of Depression Risk among Teens

By John Osorio

The goal is to build a classification model to predict depression risk in teenagers using social media usage and behavioral indicators.

Based on the public dataset *Teen Mental Health Dataset*
<https://www.kaggle.com/datasets/algozee/teenager-menthal-healy>

1.1 Imports and data loading

```
[57]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from imblearn.combine import SMOTETomek

from sklearn.metrics import accuracy_score, recall_score, precision_score, \
    f1_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay

from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

```
[58]: df = pd.read_csv("Teen_Mental_Health_Dataset.csv")
print(f'rows: {df.shape[0]}, cols: {df.shape[1]}')
```

```
rows: 1200, cols: 13
```

```
[59]: df.head(5)
```

```
[59]:   age  gender  daily_social_media_hours  platform_usage  sleep_hours  \
0    14   male                      7.9         Instagram          7.4
```

1	19	female	1.9	TikTok	8.0
2	17	female	1.3	Instagram	7.6
3	15	male	7.4	TikTok	6.9
4	15	female	4.7	Both	4.9

	screen_time_before_sleep	academic_performance	physical_activity	\
0	2.9	3.01	1.5	
1	2.9	3.22	0.8	
2	0.5	3.92	0.0	
3	1.6	3.48	0.8	
4	3.0	2.37	1.4	

	social_interaction_level	stress_level	anxiety_level	addiction_level	\
0	low	2	2	1	
1	high	8	1	10	
2	high	2	4	2	
3	medium	1	7	9	
4	medium	3	5	2	

	depression_label
0	0
1	0
2	0
3	0
4	0

```
[60]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                   1200 non-null   int64
1   gender                               1200 non-null   str
2   daily_social_media_hours             1200 non-null   float64
3   platform_usage                       1200 non-null   str
4   sleep_hours                          1200 non-null   float64
5   screen_time_before_sleep             1200 non-null   float64
6   academic_performance                 1200 non-null   float64
7   physical_activity                    1200 non-null   float64
8   social_interaction_level              1200 non-null   str
9   stress_level                         1200 non-null   int64
10  anxiety_level                        1200 non-null   int64
11  addiction_level                      1200 non-null   int64
12  depression_label                     1200 non-null   int64
dtypes: float64(5), int64(5), str(3)
memory usage: 140.4 KB
```

```
[61]: df.describe().T
```

```
[61]:
```

	count	mean	std	min	25%	50%	\
age	1200.0	15.928333	2.021947	13.0	14.0	16.00	
daily_social_media_hours	1200.0	4.536667	2.029599	1.0	2.8	4.50	
sleep_hours	1200.0	6.449417	1.442677	4.0	5.2	6.50	
screen_time_before_sleep	1200.0	1.740333	0.716660	0.5	1.1	1.80	
academic_performance	1200.0	2.990383	0.576758	2.0	2.5	2.99	
physical_activity	1200.0	1.014500	0.582185	0.0	0.5	1.00	
stress_level	1200.0	5.445833	2.903290	1.0	3.0	5.00	
anxiety_level	1200.0	5.636667	2.859453	1.0	3.0	6.00	
addiction_level	1200.0	5.565000	2.830627	1.0	3.0	6.00	
depression_label	1200.0	0.025833	0.158704	0.0	0.0	0.00	

	75%	max
age	18.00	19.0
daily_social_media_hours	6.30	8.0
sleep_hours	7.60	9.0
screen_time_before_sleep	2.40	3.0
academic_performance	3.48	4.0
physical_activity	1.50	2.0
stress_level	8.00	10.0
anxiety_level	8.00	10.0
addiction_level	8.00	10.0
depression_label	0.00	1.0

```
[62]: df.isna().sum()
```

```
[62]:
```

age	0
gender	0
daily_social_media_hours	0
platform_usage	0
sleep_hours	0
screen_time_before_sleep	0
academic_performance	0
physical_activity	0
social_interaction_level	0
stress_level	0
anxiety_level	0
addiction_level	0
depression_label	0
dtype:	int64

```
[63]: print("Duplicated rows: ", df.duplicated().sum())
```

```
Duplicated rows: 0
```

1.2 2. Exploratory Data Analysis

Extracting relevant columns

```
[64]: #target column

target_col = 'depression_label'

#categorical
cat_col = df.select_dtypes(include='object').columns.tolist()

#numerical
num_col = df.select_dtypes(include='number').columns.tolist()

if target_col in num_col:
    num_col.remove(target_col)

print(f"num_col: {num_col} \ncat_col: {cat_col}")
```

```
num_col: ['age', 'daily_social_media_hours', 'sleep_hours',
'screen_time_before_sleep', 'academic_performance', 'physical_activity',
'stress_level', 'anxiety_level', 'addiction_level']
cat_col: ['gender', 'platform_usage', 'social_interaction_level']
```

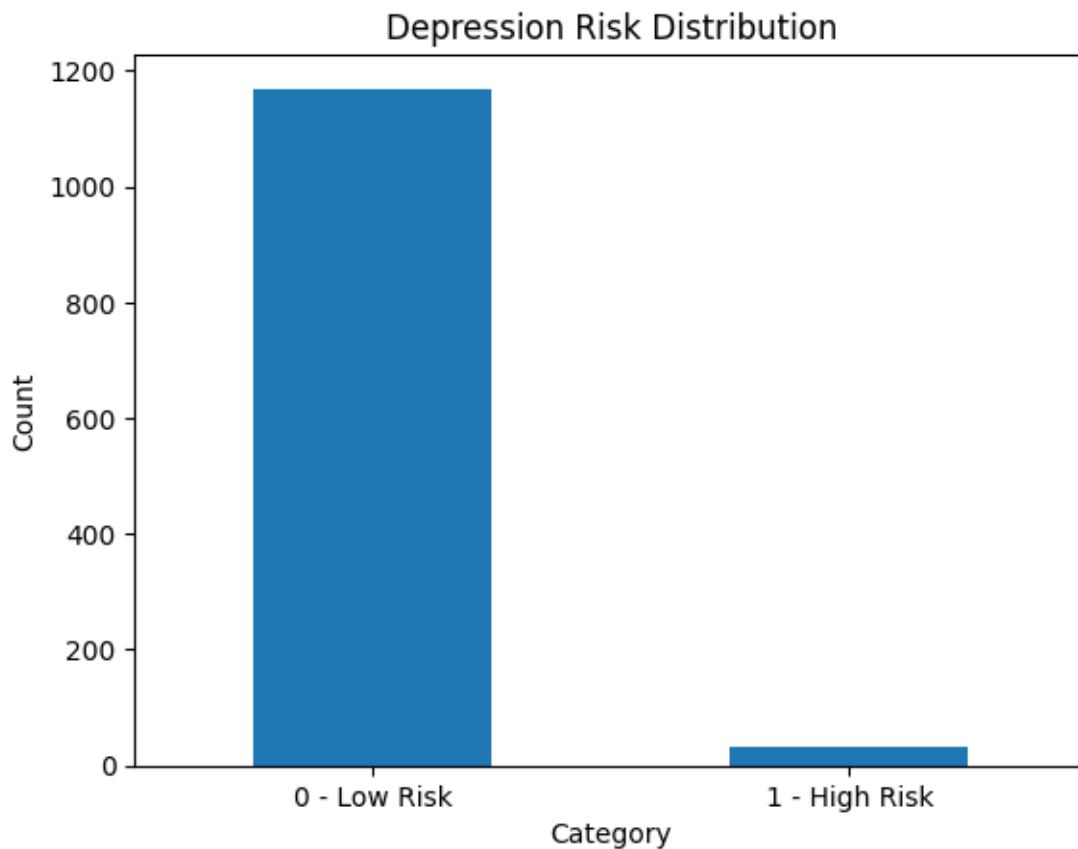
C:\Users\ASUS\AppData\Local\Temp\ipykernel_36224\1048666569.py:6:

Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning. See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
cat_col = df.select_dtypes(include='object').columns.tolist()
```

Target column distribution

```
[65]: ax = df[target_col].value_counts().sort_index().plot(kind='bar')
ax.set_xticklabels(["0 - Low Risk", "1 - High Risk"], rotation=0)
plt.title("Depression Risk Distribution")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

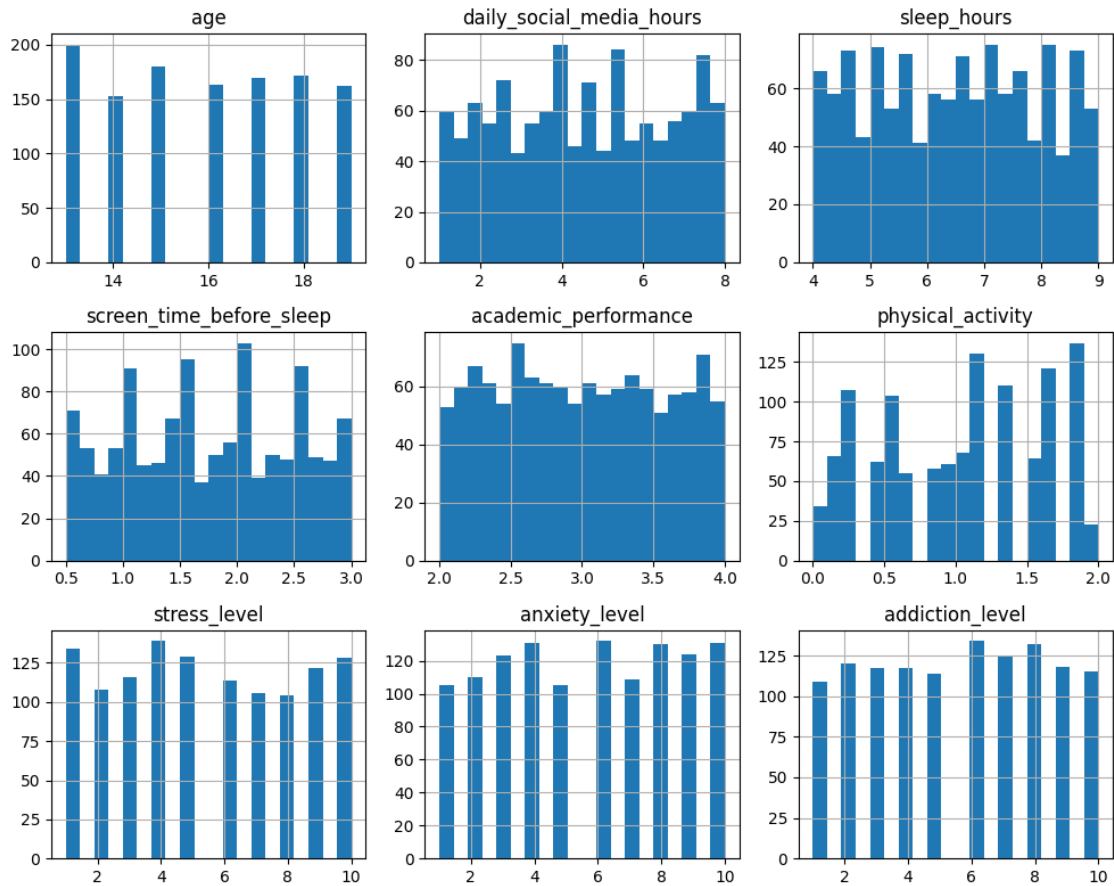


The dataset is imbalanced towards the first category, low risk. This has to be taken into account during the classification process.

Numerical columns distributions

```
[66]: df[num_col].hist(figsize=(10, 8), bins=20)

plt.tight_layout()
plt.show()
```



Relationship between target and important features

```
[67]: features = num_col.copy()
      features.remove("age")
      features.remove("academic_performance")

[68]: fig, axes = plt.subplots(3, 3, figsize=(10, 10))

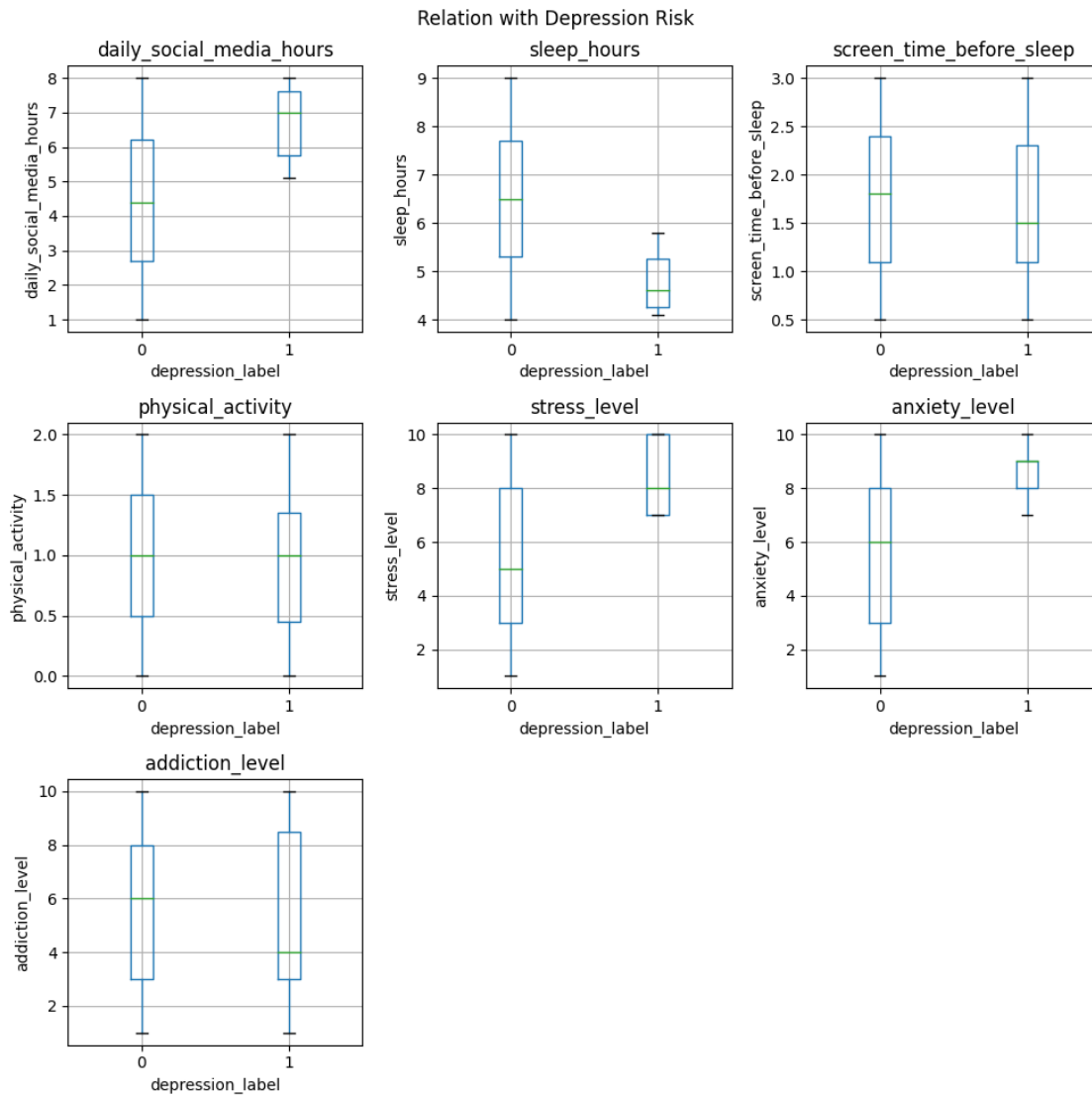
      axes = axes.flatten()

      for i, col in enumerate(features):
          df.boxplot(column=col, by=target_col, ax=axes[i])
          axes[i].set_ylabel(f"{col}")

      for j in range(i + 1, len(axes)):
          fig.delaxes(axes[j])

      plt.suptitle("Relation with Depression Risk")
      plt.tight_layout()
```

```
plt.show()
```

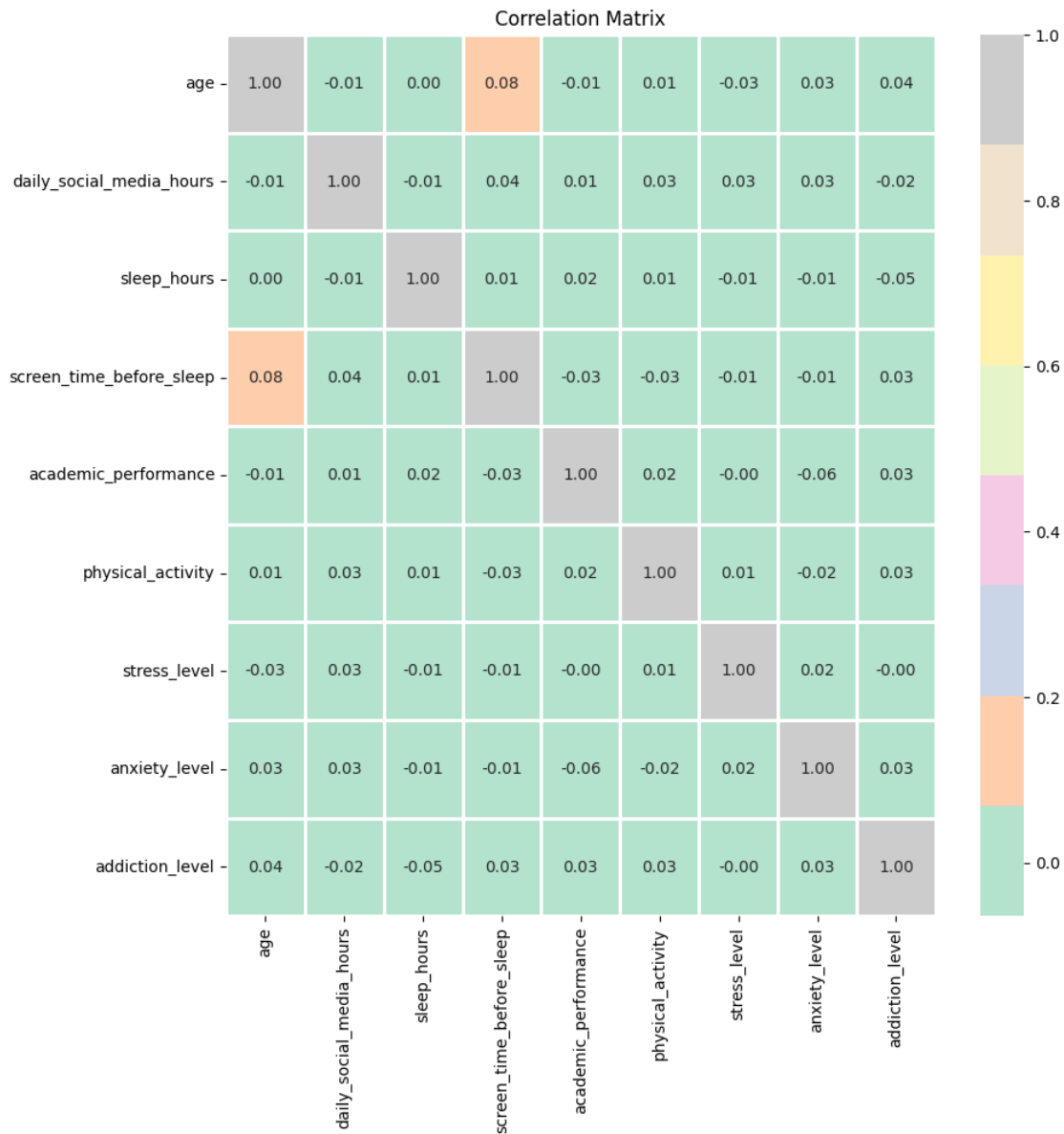


Correlation between variables

```
[69]: plt.figure(figsize=(10, 10))

sns.heatmap(df[num_col].corr(), annot=True, fmt='.2f', cmap='Pastel2',
            linewidths=2)

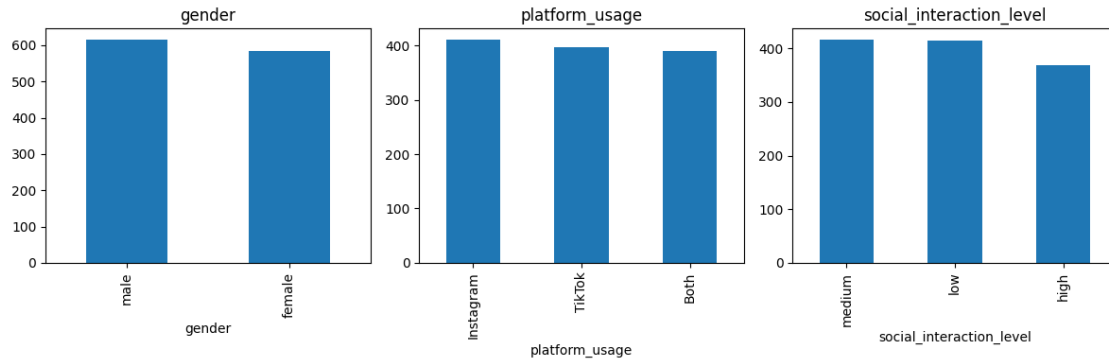
plt.title('Correlation Matrix')
plt.show()
```



Distribution of categorical features

```
[70]: fig, axes = plt.subplots(1, 3, figsize=(12, 4))
      axes = axes.flatten()

      for i, col in enumerate(cat_col):
          df[col].value_counts().plot(kind='bar', ax=axes[i])
          axes[i].set_title(col)
      plt.tight_layout()
      plt.show()
```



Relation with target

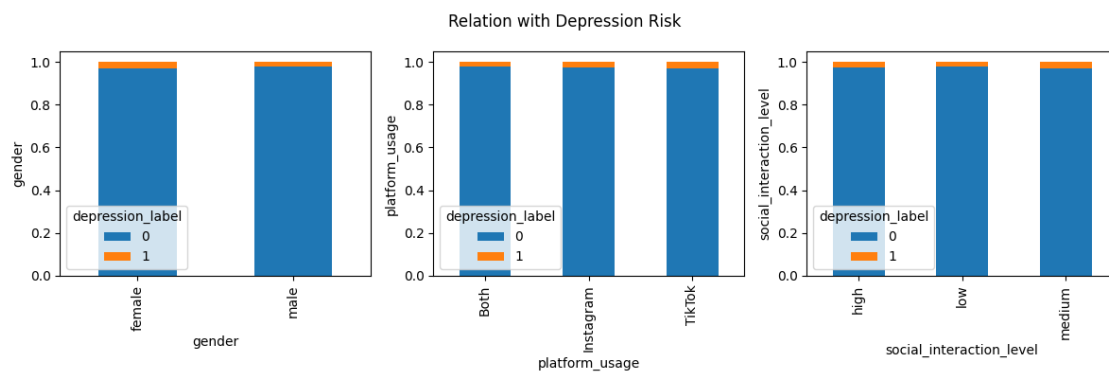
```
[71]: fig, axes = plt.subplots(1, 3, figsize=(12, 4))

axes = axes.flatten()

for i, col in enumerate(cat_col):
    pd.crosstab(df[col], df[target_col], normalize='index').plot(kind='bar',
    stacked=True, ax=axes[i])
    axes[i].set_ylabel(f"{col}")

for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle("Relation with Depression Risk")
plt.tight_layout()
plt.show()
```



1.3 3. Data Preprocessing

Label encoding for categories and standard scaling for numerical columns

```
[72]: X = df.drop(target_col, axis=1)
      y = df[target_col]
```

```
[73]: le = LabelEncoder()
      for col in cat_col:
          X[col] = le.fit_transform(X[col])
```

```
[74]: scaler = StandardScaler()
      X[num_col] = scaler.fit_transform(X[num_col])
```

```
[75]: X.head(5)
```

```
[75]:
```

	age	gender	daily_social_media_hours	platform_usage	sleep_hours	\
0	-0.954099	1	1.657833	1	0.659177	
1	1.519796	0	-1.299649	2	1.075244	
2	0.530238	0	-1.595397	1	0.797866	
3	-0.459320	1	1.411376	2	0.312455	
4	-0.459320	0	0.080509	0	-1.074435	

	screen_time_before_sleep	academic_performance	physical_activity	\
0	1.618830	0.034026	0.834276	
1	1.618830	0.398282	-0.368593	
2	-1.731436	1.612470	-1.743301	
3	-0.195897	0.849266	-0.368593	
4	1.758424	-1.076088	0.662437	

	social_interaction_level	stress_level	anxiety_level	addiction_level
0	1	-1.187367	-1.272335	-1.613389
1	0	0.880116	-1.622199	1.567444
2	0	-1.187367	-0.572609	-1.259963
3	2	-1.531947	0.476980	1.214018
4	2	-0.842786	-0.222746	-1.259963

Splitting the dataset for training and testing, while taking into account the imbalance “stratify=y”

```
[76]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
      ↪random_state=1234, stratify=y)
```

Using SMOTE to handle the imbalanced dataset

```
[77]: smote = SMOTETomek(random_state=1234)

      X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

1.4 4. Classification Models

Function to evaluate the metrics of a given model

```
[78]: def eval_model(clf_model, X_train, X_test, y_train, y_test):
    clf_model.fit(X_train, y_train)
    y_pred = clf_model.predict(X_test)

    if hasattr(clf_model, "predict_proba"):
        y_prob = clf_model.predict_proba(X_test)[:, 1]
        roc_auc = roc_auc_score(y_test, y_prob)
    else:
        roc_auc = None

    model_acc = accuracy_score(y_test, y_pred)
    model_pre = precision_score(y_test, y_pred)
    model_f1 = f1_score(y_test, y_pred)
    model_rec = recall_score(y_test, y_pred)
    model_cm = confusion_matrix(y_test, y_pred)

    return {'Accuracy': model_acc, 'Precision': model_pre, 'Recall': model_rec,
    ↪ 'F1_Score': model_f1, 'ROC_AUC': roc_auc}, model_cm
```

We will test four different classifiers

```
[79]: Models = {
    'DecisionTreeClassifier': DecisionTreeClassifier(max_depth = 2,
    ↪ random_state = 1234, class_weight="balanced"),
    'LogisticRegression': LogisticRegression(max_iter=10000, random_state =
    ↪ 1234),
    'RandomForestClassifier': RandomForestClassifier(max_depth = 2,
    ↪ random_state = 1234, class_weight="balanced"),
    'XGBClassifier': XGBClassifier(learning_rate=0.1)
}
```

```
[80]: results = []
cmatrices = []
for name, model in Models.items():
    metrics, cm = eval_model(model, X_train_smote, X_test, y_train_smote, y_test)
    metrics = {'Model': name, **metrics}
    results.append(metrics)
    cmatrices.append(cm)
models_metrics = pd.DataFrame(results)
```

Confusion Matrices for all the models

```
[81]: fig, axes = plt.subplots(2, 2, figsize=(10, 10))

axes = axes.flatten()

for i, model_name in enumerate(Models.keys()):
```

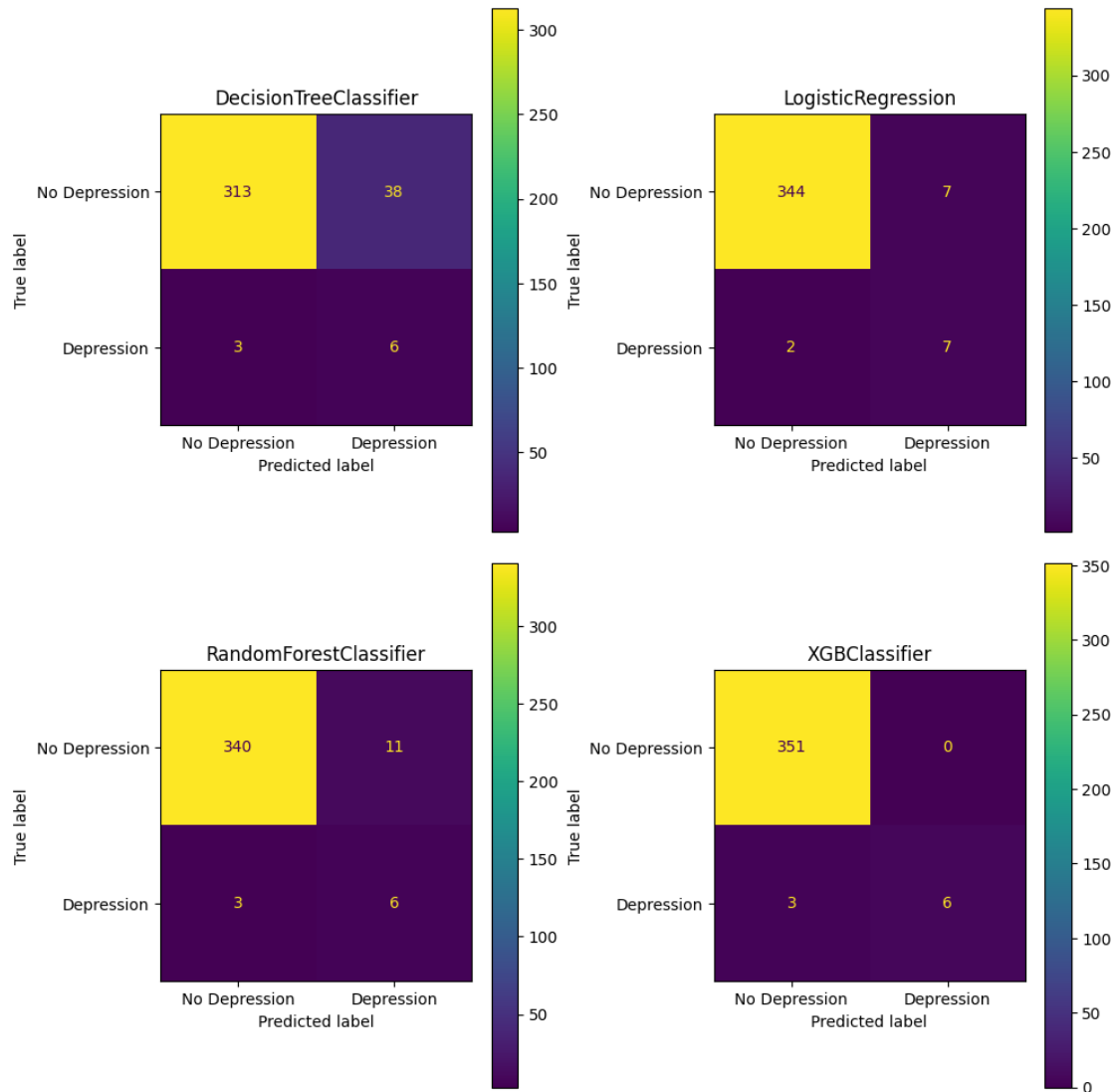
```

ConfusionMatrixDisplay(
    confusion_matrix=cmatrices[i],
    display_labels=["No Depression", "Depression"]
).plot(ax=axes[i])

axes[i].set_title(f"{model_name}")

plt.tight_layout()
plt.show()

```



Important metrics for classification

```
[82]: models_metrics.sort_values(by="F1_Score", ascending=False)
```

[82]:

	Model	Accuracy	Precision	Recall	F1_Score	ROC_AUC
3	XGBClassifier	0.991667	1.000000	0.666667	0.800000	0.993985
1	LogisticRegression	0.975000	0.500000	0.777778	0.608696	0.983856
2	RandomForestClassifier	0.961111	0.352941	0.666667	0.461538	0.968661
0	DecisionTreeClassifier	0.886111	0.136364	0.666667	0.226415	0.789332

2 5. Conclusions

Depression risk among teens was predicted using four different models for classification. The models show varying performance, with XGBClassifier outperforming the rest of the models, with a precision of 1.00, an F1-Score of 0.8 and ROC AUC score of 0.99.

Decision tree based models performed poorly on this dataset, with lower precision and overall performance compared to the other models.